

CNT4713 File Transfer Project

Overview

Disclaimer: If you did not complete the chat project you may wish to do so before starting this project. However, your Chat Project grade will remain the same. All commands, flow, and operation from the Chat project are still needed as the base for connections and must remain.

In this project, you will extend the Chat Project to include file upload capabilities. These capabilities will work similarly to the File Transfer Protocol (FTP), but are not the same. It would be helpful to review how FTP works using the references at the end of this document. You must create your own socket, and cannot use any existing FTP library. You must use the **Python** programming language.

Constraints

- Use socket programming only.
- Use TCP sockets.
- Programming language: Python
- No external messaging frameworks.
- Client and server must run as separate processes.
- Server file **must be named**: server.py
- Client file **must be named**: client.py
- You must explicitly follow the input and output guidelines.

An autograder will grade the code submissions. This autograder executes pre-written versions of the client and server separately with your submission (your client + known server, your server + known client). You must therefore explicitly name the files as instructed. Also, you must explicitly follow the commands, flow, input, and output given. **From a runtime perspective, the usernames, ports, IP addresses, messages, and file names are all variable.** The grader will use their own values for these settings and the expected out **must** match. All submissions which do not match will received a **zero**.

The Server

The server will now allow users to upload files and create directories on the server for other users to navigate and download. The following table outlines the **new** commands the server will accept. All commands will return status code **200** when the server executes the command successfully, and **500** when the command fails.

Command	Example
stor <filename>	stor myfile.txt
retr <filename>	retr myfile.txt
list	list
dele <filename>	dele myfile.txt

Responses from the server **must all have the following format** (where necessary):

Status code

<EMPTY LINE>

Data section if required

Further explanation on the commands is found below:

1. The server will be started via command line and login will proceed exactly the same as in the Chat project.
2. The server will listen for the following **new commands** (in bold) from the client and:
 - i. **stor** – Used to store (that is, put) a file into the current directory of the server. The server saves the filename and uses it to create a file on the server using the incoming data on the DATA PORT. Only the status code is expected on response.
 - ii. **retr** – Used to retrieve (that is, get) a file from the current directory of the server. This command causes the server to send the requested file over the data connection. On failure, only the failing status code is expected on response.
 - iii. **list** – Used to ask the server to send back a list of all the files in the current remote directory. A comma separated list of files is sent over the data connection rather than the control TCP connection. On failure, only the failing status code is expected on response. Example server response:


```
200

myfile.txt, test.txt
```
 - iv. **dele** – Used to delete a file in the current directory of the server. Only the status code is expected on response.

The Client

The following table outlines the commands the client will send. All commands will return **200** when the server executes the command successfully, and **500** when the command fails.

Further explanation on the commands is found below:

1. The client will be started via command line and login will proceed exactly the same as in the Chat project.
2. Upon successfully login to the server, the following commands are sent over the CONTROL PORT with responses on the DATA PORT:
 - i. **list** – The requests the list of files currently on the server. A comma separated list of files is returned.
 - ii. **stor** – The client requests server to save a file providing the filename. The contents of the file is sent over the DATA PORT.
 - iii. **retr** – The client requests the server to send a file identified by filename. The server send the contents of the file over the DATA PORT.
 - iv. **dele** – The client requests the server to delete the file identified by file name. Only the status code is returned.

Output

Explicitly use the following output guideline template. ">" at the start of a line indicates a stage requiring user input or intervention:

1. The Server

```
> python server.py 8991
Starting server...
Creating server socket
Awaiting connections...
Connection requested. Creating data socket
Login requested by: alice
Connection requested. Creating data socket
```

List requested by alice. Sending files.

Stor myfile.txt requested by alice
STOR complete

List requested by alice. Sending files.

Retr requested by alice. Sending file: myfile.txt
File sent.

Dele requested by alice. Deleting file: myfile.txt
Delete complete

Quit requested by alice

2. The Sending Client

```
> python client.py
```

```
Starting client...
```

```
> connect 192.168.254.253 8991
```

```
200 status code received. Starting data connection on port  
<DATA PORT>
```

```
> login alice
```

```
200 status code received. Login successful
```

```
> list
```

```
200 status code received. Files: test.txt
```

```
> stor myfile.txt
```

```
200 status code received. File Sent.
```

```
> list
```

```
200 status code received. Files: myfile.txt, test.txt
```

```
> retr test.txt
```

```
File retrieved.
```

```
> dele test.txt
```

```
200 status code received. File deleted.
```

```
> quit
```

```
200 status code received.
```

Submission

1. A readme.txt file with the Group Member Names and IDS.
2. A copy of your final code to the LMS. (**Source code only (.py files). No zips. No IDE project files**). Do **not** submit zip files to file share site or links to zips at file share sites (including Google Drive). **Zero will be awarded without chance for review.**
3. A video recording (screen recording) of the execution of your team's project. The recording must show your application being executed for each graded part of the project. Narrate the video stating what is being shown.
4. Use Wireshark to show the messages being sent between the client(s) and the server. Ensure to show and highlight the message formats stated in the requirements. Wireshark will be a portion of the video grade.

You may upload the video to YouTube or Google Drive and submit the link to the LMS. **The link must be viewable at time of grading. Zero with no review will be awarded if it is not viewable at time of grading.**

Grading

The table below shows how the project will be graded. **Grading shown in the LMS and instructions from your instructor override all grading shown below.**

Item	Percentage
Server connect and login	3
Server list command	6
Server stor command	10
Server retr command	10
Server dele command	6
Client connect and login	3
Client list command	6
Client stor command	10
Client retr command	10
Client dele command	6
video	30

Code plagiarism will be reported for academic dishonesty.

References:

- <https://medium.com/@CHICHEEE/build-a-simple-file-transfer-system-using-python-network-programming-project-c9962cf238c0>
- <https://www.cs.colostate.edu/helpdocs/ftp.html>